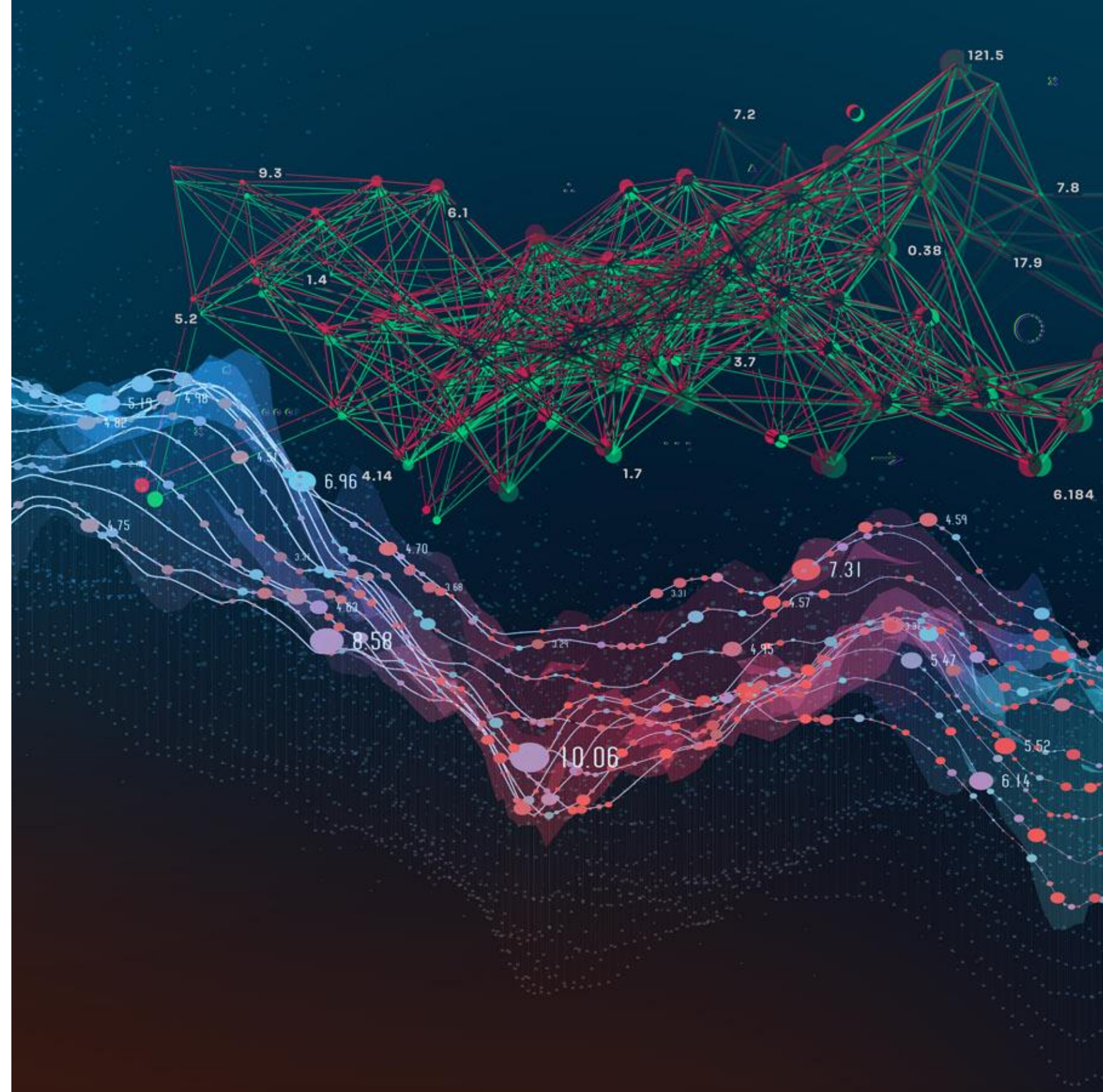


Realistic DRAM Timing for HLS: Closing the Memory Fidelity Gap with Ramulator-Backed Co-Simulation

July 6, 2026

Jesun Firoz, Giovanni Gozzi, Michele Fiorito,
Joseph Manzano, Vito Giovanni Castellana,
Ankur Limaye, Oceane Bel, Fabrizio Ferrandi,
Kevin Barker, **Antonino Tumeo**



High-Level Synthesis (HLS)

Automatic translation of a **behavioral** specification of a circuit to a hardware description language (HDL) implementation

- **Input:** any high-level “programming” language such as C/C++, SystemC, Fortran, DSLs...
- **Output:** a description of the circuit in a hardware language (Verilog, VHDL, ...)
- **Typical Architectural Model:** Finite State Machine with Datapath (FSMD)

Objectives: increase productivity in hardware design, by writing specifications in high-level languages rather than register-transfer level (RTL) code directly. Consequently, we can perform design-space exploration and optimization much faster.

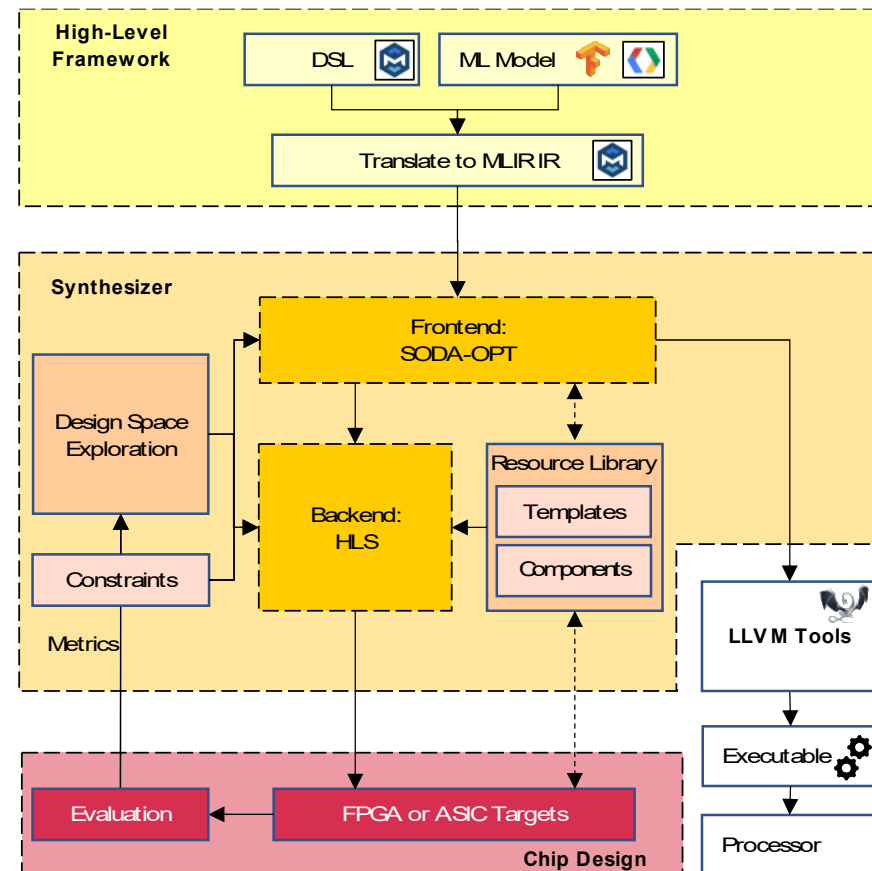
Verification & Simulation in HLS

- The generated RTL is usually verified with cycle-by-cycle simulation
- Advanced HLS tools can automatically generate testbenches and perform co-simulation (for example integrating with Verilator)
- However, external memory is usually modeled as a fixed-latency slave. This hides all typical behaviors that instead lead to variable latency: row-buffer hits and conflicts, bank interference, refresh effects, memory-controller queuing, out-of-order completion behavior
- **Result:** simulation might allow to verify functional behavior, but the cycle performance estimates for memory-bound accelerators can be systematically wrong

Why this Matters in HLS?

- HLS can enable quicker exploration of design choices before hardware implementation
- However, memory sensitive decisions depend on realistic memory behaviors and latencies:
 - Tiling, where and how to allocate and partition data, how to pipeline and overlap between memory operations and computer
- Fixed-delay models can work only for simple designs and well-structured applications
- Adding hardware in the loop (e.g., deploying an FPGA) to measure and optimize can add days per design iteration

SODA Synthesizer: Overview



[M. Minutoli, V. G. Castellana, C. Tan, J. Manzano, V. Amaty, A. Tumeo, D. Brooks, G-Y. Wei: SODA: a New Synthesis Infrastructure for Agile Hardware Design of Machine Learning Accelerators. ICCAD 2020: 98:1-98:7]

[J. Zhang, N. Bohm Agostini, S. Song, C. Tan, A. Limaye, V. Amaty, J. Manzano, M. Minutoli, V. G. Castellana, A. Tumeo, G-Y. Wei, D. Brooks: Towards Automatic and Agile AI/ML Accelerator Design with End-to-End Synthesis. ASAP 2021: 218-225]

- A modular, multi-level, interoperable, extensible, **open-source hardware compiler** from **high-level programming frameworks to silicon**
- Compiler-based frontend, leveraging the MultiLevel Intermediate Representation (MLIR)
- **Compiler-based backend**, leveraging state-of-the-art High-Level Synthesis (HLS) techniques, as well as a Coarse-Grained Reconfigurable Array (CGRA) generator
- Generates **synthesizable Verilog** for a variety of targets, from Field Programmable Gate Arrays (FPGAs) to Application Specific Integrated Circuits (ASICs)
- Optimizations at all levels are performed as **compiler optimization** passes

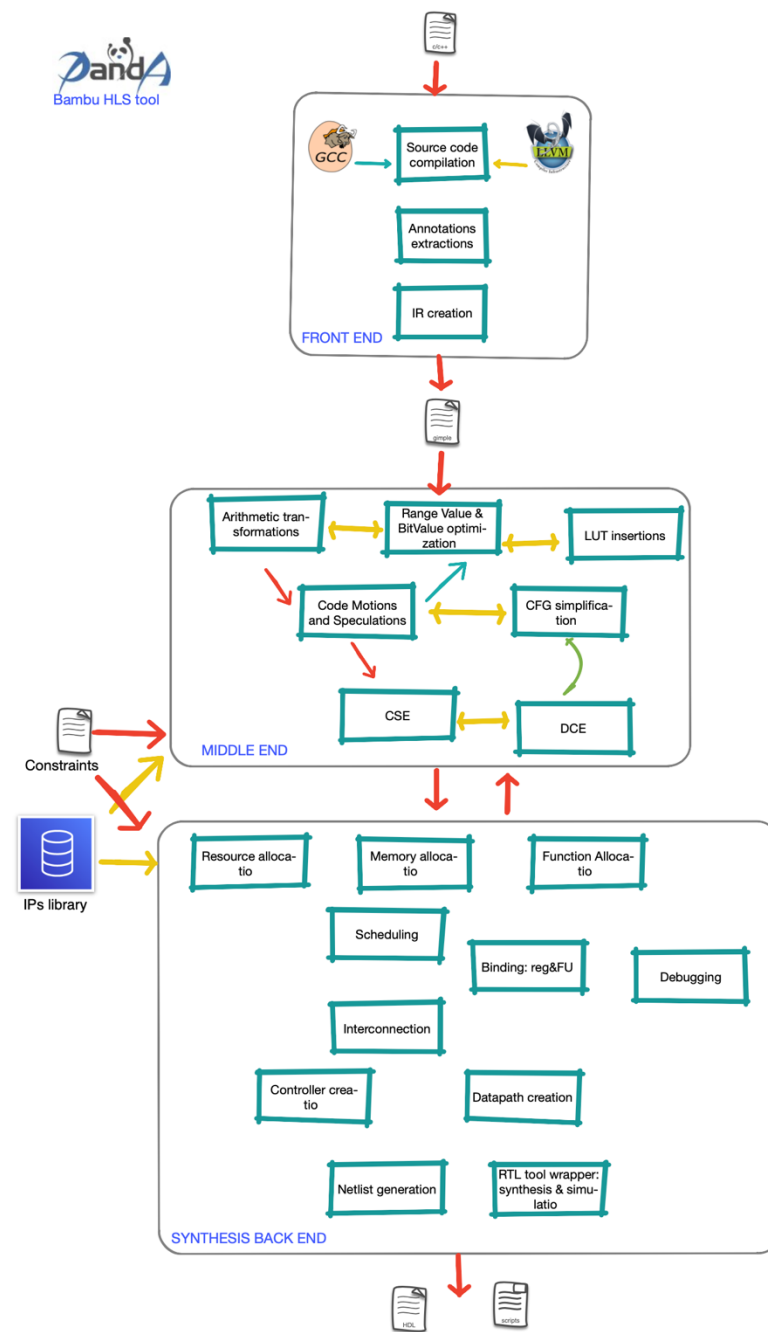
[N. Bohm Agostini, S. Curzel, J. Zhang, A. Limaye, C. Tan, V. Amaty, M. Minutoli, V.G. Castellana, J. Manzano, A. Tumeo: Bridging Python to Silicon: The SODA Toolchain. IEEE Micro Magazine 2022]

[N. Bohm Agostini, S. Curzel, V. Amaty, C. Tan, M. Minutoli, V. G. Castellana, J. Manzano, D. Kaeli, A. Tumeo : An MLIR-based Compiler Flow for System-Level Design and Hardware Acceleration. ICCAD 22]

[Fabrizio Ferrandi, Vito Giovanni Castellana, Serena Curzel, Pietro Fezzardi, Michele Fiorito, Marco Lattuada, Marco Minutoli, Christian Pilato, Antonino Tumeo: Invited: Bambu: an Open-Source Research Framework for the High-Level Synthesis of Complex Applications. DAC 2021: 1327-1330]

HLS Backend: PandA- Bambu

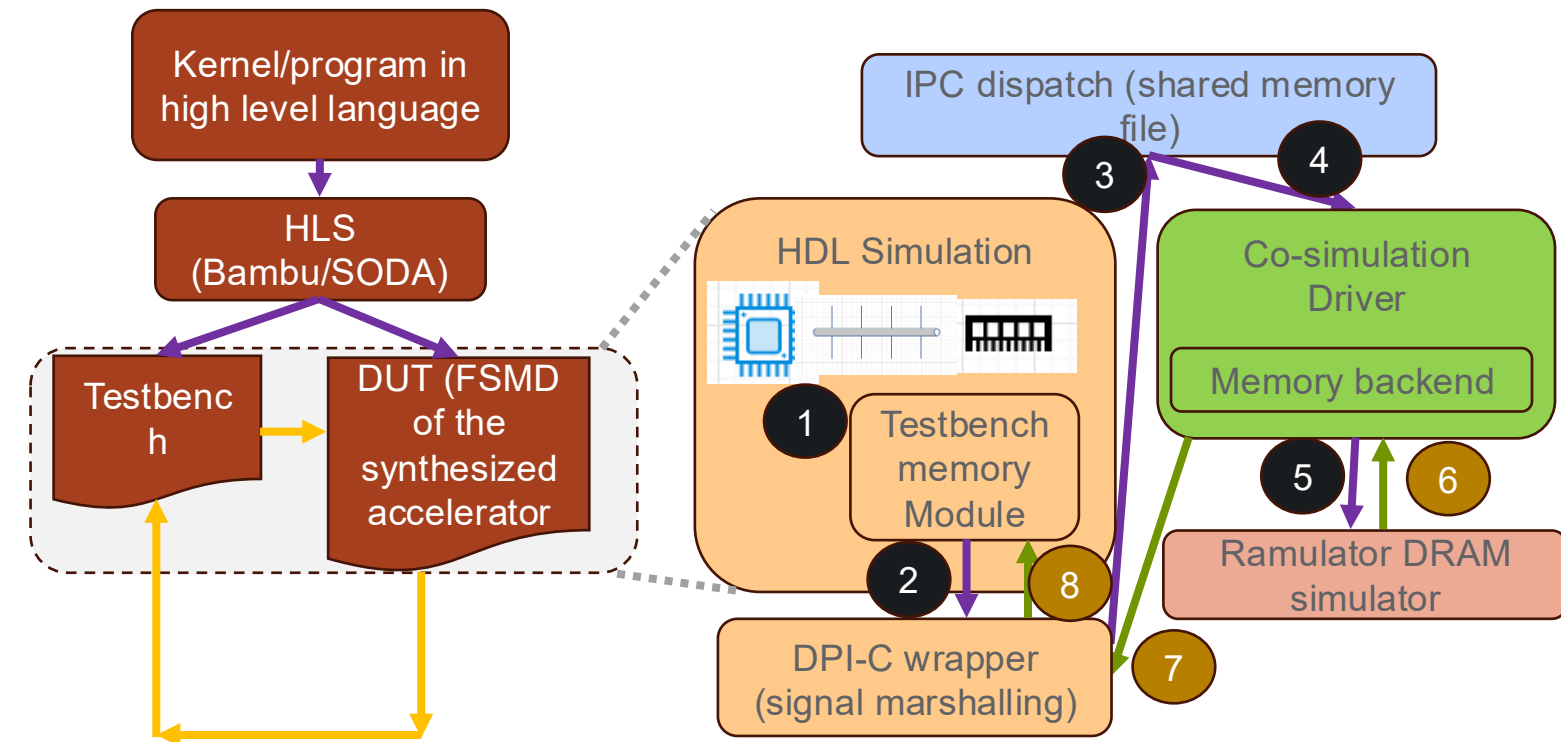
- State-of-the-art opensource High-Level Synthesis Tool
- Interfaces with different "front-end" compilers: GCC, LLVM, Xilinx LLVM frontend, MLIR-based frontends...
- Integrates verification and component based design
- Supports different synthesis backend: FPGAs (AMD/Xilinx, Intel/Altera, Lattice, NanoXplore...) and ASICs (opensource, commercial)
- Pluggable resource library, with accurate models to drive the HLS algorithms
- "Parallel" accelerator synthesis methodologies, including temporal and spatial multithreading



<https://panda.dei.polimi.it>

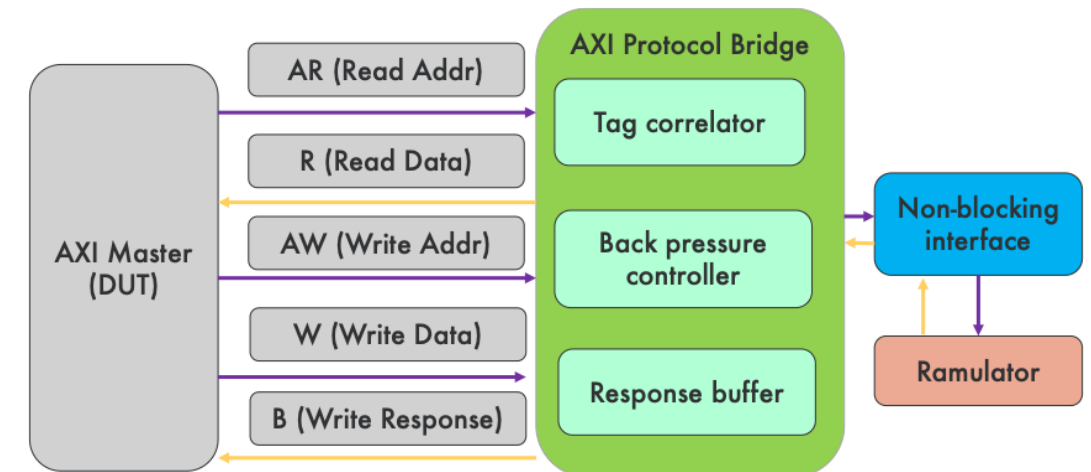
Main Idea

- Replace Bambu's fixed-latency memory backend with Ramulator v2.0
- Preserve the HLS-generated RTL/testbench flow
- Add a protocol-faithful AXI4 bridge between:
 - HLS bus handshakes
 - Ramulator's callback-driven DRAM timing model
- **Goal:** make memory timing realistic during simulation, without needing deployment on FPGA



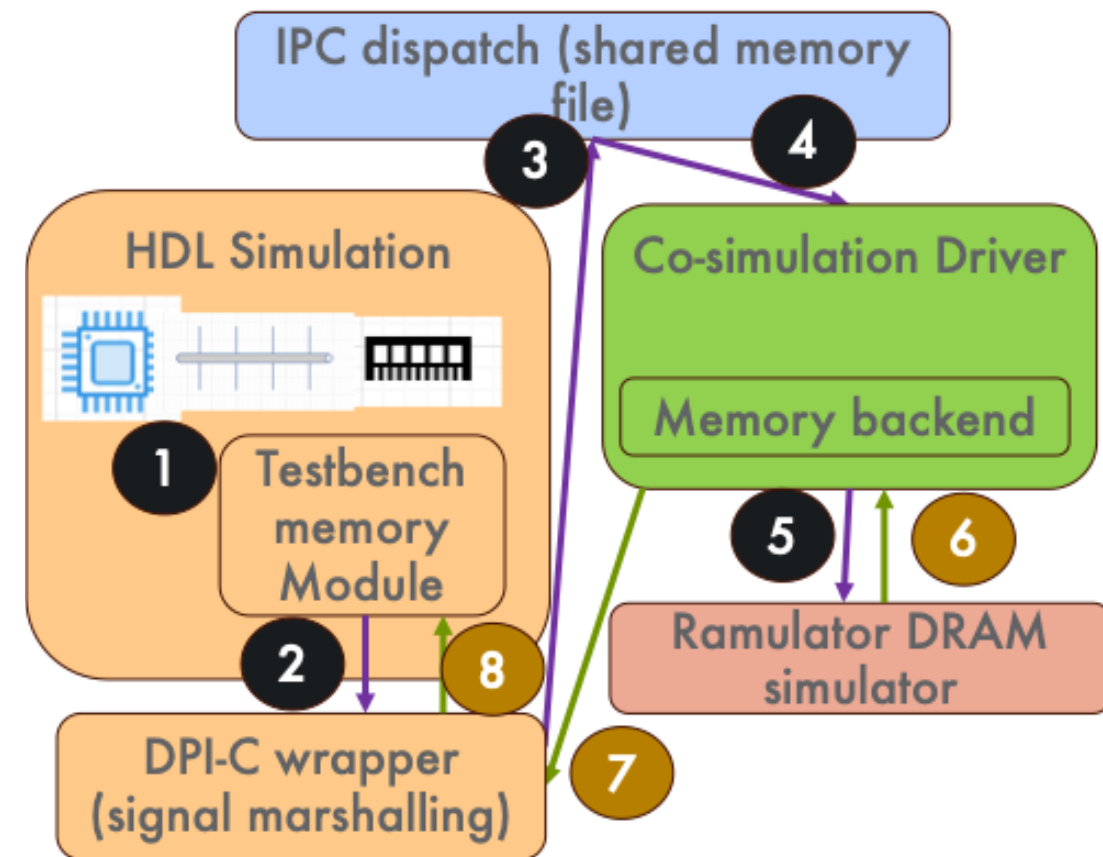
What Makes this Non-Trivial?

- AXI4 is split-transaction:
 - request and response are decoupled
 - multiple outstanding requests are allowed
 - responses may return out of order
- Ramulator is non-blocking and callback-driven
- HLS testbench logic expects cycle-by-cycle protocol semantics
- The bridge must preserve:
 - AXI valid/ready handshakes
 - transaction IDs
 - back-pressure
 - response stability
 - functional memory correctness



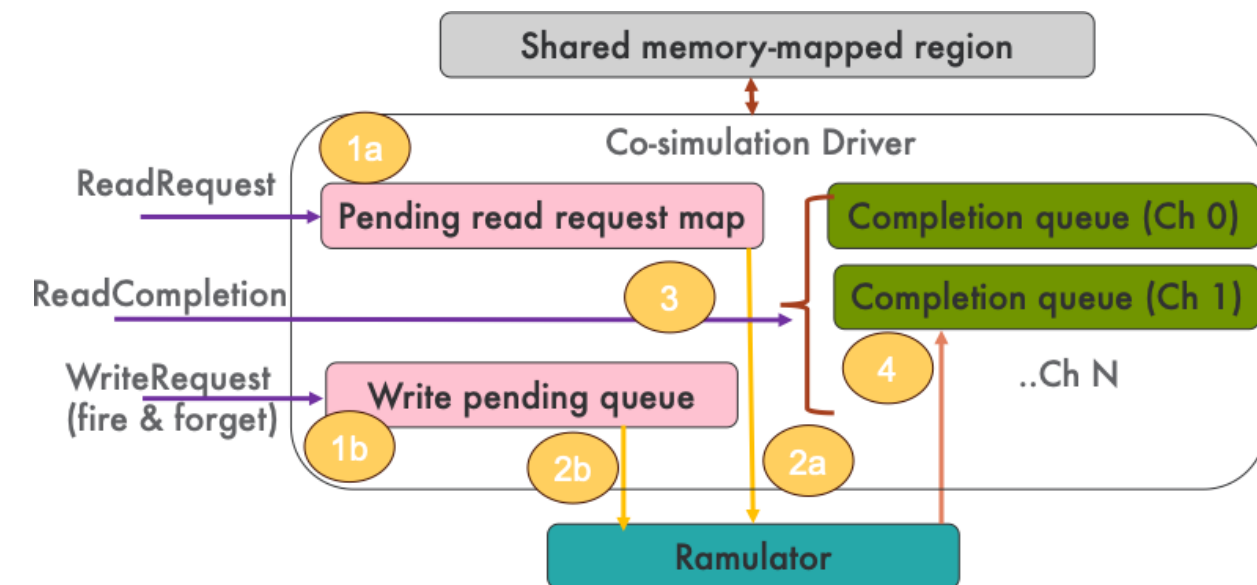
Co-Simulation Architecture

- Two cooperating processes:
 - HDL simulator: DUT + generated testbench memory modules.
 - C++ driver: host-side memory state, test data, verification, Ramulator backend.
- Communication path:
 - Verilog testbench
 - Direct Programming Interface - C (DPI-C) wrapper
 - Interprocess Communication (IPC) dispatch over shared memory
 - C++ interface object
 - memory backend
- Backend is pluggable:
 - default fixed-delay backend
 - cycle-accurate Ramulator backend



Non-Blocking Request / Completion Interface

- **Key abstraction:** separate request submission from response delivery
- **Interface:**
 - `ReadRequest(addr, size, channel) → tag`
 - `ReadComplete(channel) → tag, data`
 - `WriteRequest(data, addr, size) → status`
 - `AdvanceClock(n)`
- Reads complete only when Ramulator says the DRAM request completes
- Writes update shared memory immediately for correctness, while also submitting timing events

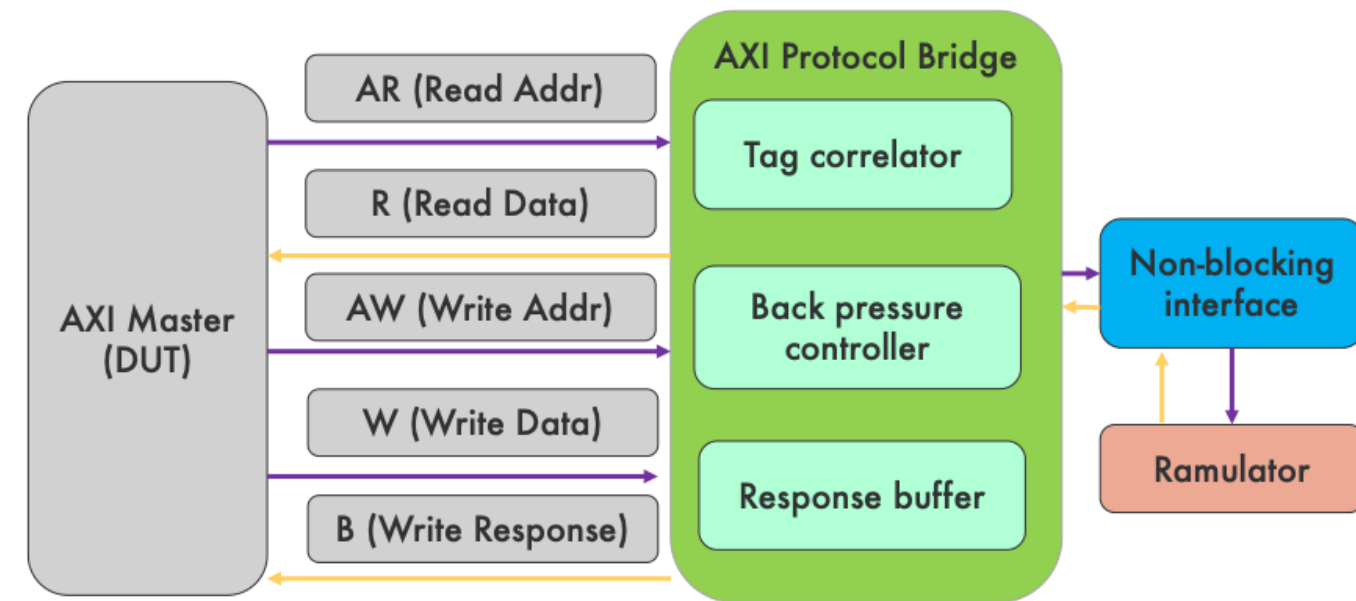


Clock Coordination and Frequency Scaling

- Accelerator and DRAM run in different timing domains
- For accelerator frequency f_{accel} and DRAM command frequency f_{DRAM} :
 - $R = f_{\text{DRAM}} / f_{\text{accel}}$
- In the evaluation:
 - accelerator: 200 MHz
 - DDR4-2400R command clock: 1200 MHz
 - $R = 6$
- Each accelerator cycle advances Ramulator by the scaled DRAM ticks
- Idle periods can be bulk-advanced to reduce DPI/IPC overhead

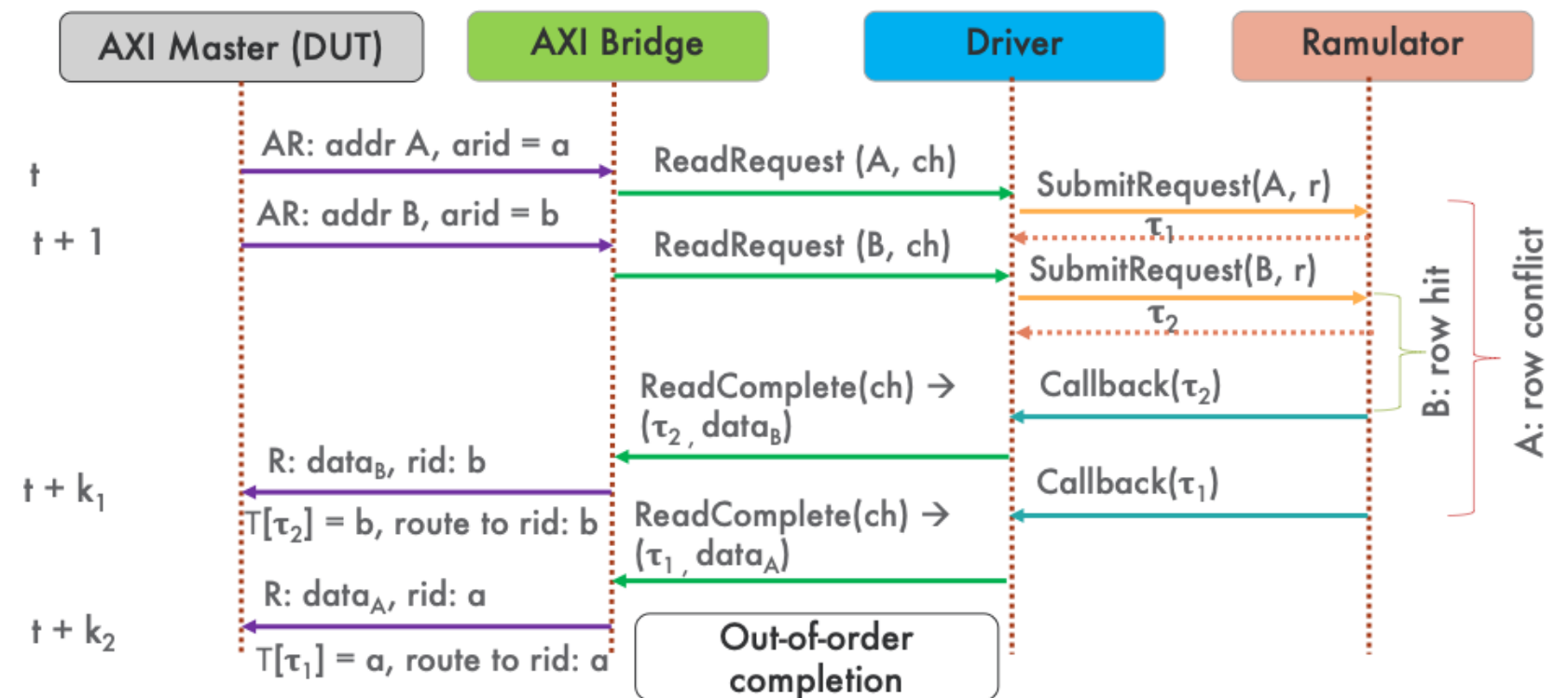
Protocol-Faithful AXI4 Bridge

- The bridge has three essential mechanisms:
 - **Tag correlator**: maps Ramulator tags back to AXI transaction IDs
 - **Back-pressure controller**: stalls AR/AW when bridge queues or Ramulator buffers are full
 - **Response buffer**: holds completed responses until AXI ready completes the handshake
- This preserves AXI semantics while allowing realistic DRAM timing.



Why Out-of-Order Completion Matters

- Example:
 - Request A arrives first but causes a row conflict
 - Request B arrives second but hits an open row
 - Ramulator completes B before A.
- Fixed-delay memory cannot represent this
- AXI IDs preserve correctness:
 - B returns with `rid = b`
 - A returns with `rid = a`



Experimental Setup

- Target: Xilinx Alveo U250
- Accelerator clock: 200 MHz
- DRAM model: DDR4-2400R, matched to the board
- Simulation infrastructure leverages Bambu co-simulation infrastructure with Verilator, Ramulator-v2.0
- Benchmarks:
 - Sequential: 13 PolyBench kernels, 3 MachSuite kernels. Architecture generated is a standard FSMD
 - Parallel: triangle count and page rank from the Graph Application Platform (GAP) Benchmark Suite. Architecture generated is a multithreaded parallel design through Bambu's OpenMP flow (SPARTA)
- Memory models compared:
 - Bambu default fixed delay: 2 cycles
 - row-conflict fixed baseline: 8 cycles
 - Ramulator-backed cycle-accurate model
- Metric: simulated accelerator cycle count from start to done.

Serial Kernels

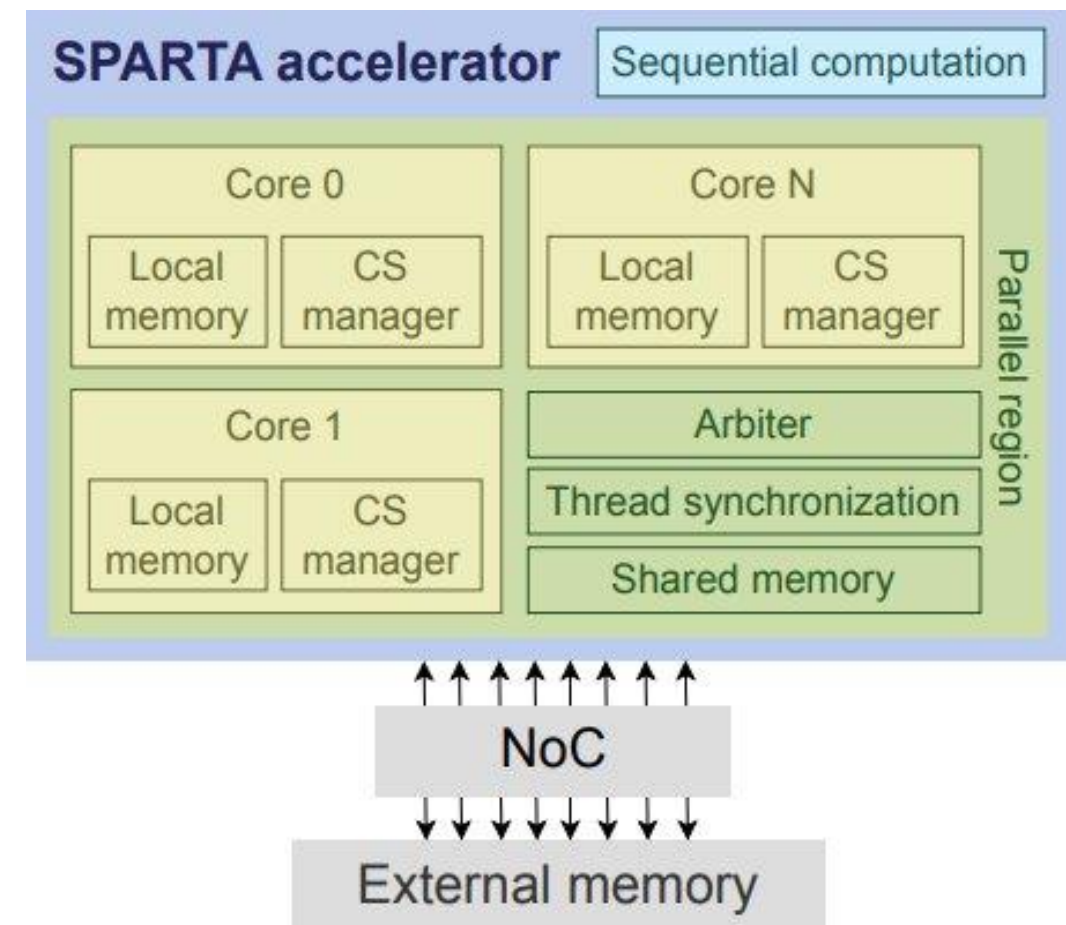
- Main results:
 - 2-cycle model under-predicts runtime
 - 8-cycle row-conflict model over-predicts many serial kernels
 - The true value depends on access pattern

Kernel	Ramulator	Fix 2c	ρ_2	Fix 8c	ρ_8
bicg	22.0k	15.1k	1.46×	35.2k	0.62×
seidel-2d	1.43M	909k	1.58×	1.63M	0.88×
gemm	228k	143k	1.60×	332k	0.69×
trisolv	12.4k	7.66k	1.62×	18.0k	0.69×
atax	38.4k	22.9k	1.68×	52.3k	0.73×
jacobi-1d	16.2k	9.56k	1.70×	23.5k	0.69×
2mm	144k	82.0k	1.76×	172k	0.84×
3mm	212k	118k	1.79×	260k	0.81×
doitgen	123k	67.6k	1.82×	149k	0.83×
mvt	30.6k	16.6k	1.84×	36.5k	0.84×
cholesky	260k	136k	1.91×	339k	0.77×
heat-3d	1.31M	630k	2.08×	1.51M	0.87×
jacobi-2d	1.31M	619k	2.11×	1.57M	0.83×
Geo. mean			1.75×		0.77×

Kernel	Ramulator	Fix 2c	ρ_2	Fix 8c	ρ_8
spmv/csr	43.8k	18.8k	2.33×	44.7k	0.98×
gemm-blocked	4.14M	2.49M	1.66×	5.84M	0.71×
sort-merge	715k	283k	2.53×	958k	0.75×

SPARTA: High-Level Synthesis of Parallel Multi-Threaded Accelerators

- A methodology that directly synthesizes OpenMP annotated code leveraging the OpenMP runtime
 - At compile time, calls to the OpenMP runtime are substituted by calls to synthesizable equivalents
- Supports a subset of OpenMP constructs:
 - Parallel for and sections
 - Reduction, shared, private and firstprivate clause
 - Critical section
 - Nested worksharing constructs
- Generates a custom architecture with spatial and temporal replication and a bufferless hot potato network on chip to connect to external memory



Parallel Kernels

- Using an RMAT-13 graph
- Configurations:
 - 1×1: serial baseline
 - 1×8: temporal multithreading
 - 8×1: spatial replication
 - 8×8: spatial × temporal parallelism
- Main result:
 - At maximum parallelism, fixed 2c and fixed 8c nearly agree
 - But both are still far from Ramulator
- Interpretation:
 - The remaining error is not per-access latency
 - It is memory-system structure: queuing, bank conflicts, refresh contention, concurrent masters

WL	$C \times T$	Ramulator	Fix 2c	ρ_2	Fix 8c	ρ_8
pr	1×1	4.96M	2.39M	2.07×	5.48M	0.91×
	1×8	2.44M	2.17M	1.13×	2.41M	1.01×
	8×1	992k	483k	2.06×	711k	1.40×
	8×8	935k	471k	1.98×	477k	1.96×
tc	1×1	7.43M	3.24M	2.29×	7.80M	0.95×
	1×8	4.13M	3.15M	1.31×	3.98M	1.04×
	8×1	2.59M	1.17M	2.21×	2.47M	1.05×
	8×8	1.75M	1.01M	1.73×	1.08M	1.62×

Conclusions

- Fixed-delay HLS memory models are insufficient for memory-bound accelerators
- Ramulator-backed co-simulation exposes:
 - row-buffer behavior
 - bank conflicts
 - refresh effects
 - memory-controller queuing
 - out-of-order completions
- Protocol fidelity matters:
 - not just “call Ramulator”
 - AXI semantics must be preserved.
- Design-space implications:
 - evaluate memory-sensitive HLS choices before hardware implementation
- Opportunities:
 - Evaluate designs with different types of memories and memory architecture
 - Feedback results into the HLS flow to enable memory-aware synthesis

This work has been supported by
The DOE ASCR Competitive
Portfolio Project ENCODE (End-to-
end Co-design) at PNNL.

Thank you

Questions:

jesun.firoz@pnnl.gov

antonino.tumeo@pnnl.gov

